TD1
Prise en main, introspections, gestion des fichiers

Thomas Sarno

Septembre - Octobre 2026

1 Introduction

Le but de ce TD est de bâtir un environnement de test pour vérifier le bon fonctionnement d'un bloc de chiffrement AES. Cet AES est limité au chiffirement de blocs de données d'une taille de 128 bits avec une clé de chiffrement ayant également une taille de 128 bits.

2 Modalités

Ce TD sera évalué sur la base d'un compte rendu à rendre sur ecampus et d'une rapide évaluation de l'avancement de votre travail à la fin de chaque séance.

2.1 Evaluations intermediaires

A la fin de chaque séance, l'enseignant verifiera l'avancement de votre travail et une partie de la note sera attribuée sur la base de cet avancement.
Les éléments qui seront pris en compte pour cette évaluation sont les suivants :

— la progression effective du travail ;
— la compréhension des manipulations réalisées ;
— l'autonomie et la capacité à justifier les choix effectués ;

2.2 Rapport final

Il vous est également demandé de rédiger un compte rendu pour ce TD dans lequel vous ferez figurer toutes les remarques pertinentes qui vous semblent nécessaires à la bonne compréhension du problème posé et de la solution apportée.

3 Prise en main

Dans un premier temps récupérez l'archive TD1.tgz sur ecampus et excécuter la commande suivante dans le répertoire de travail de votre choix :
tar -xzvf TD1.tgz

Le testbench présent dans l'archive est vide, vous devrez le compléter au fur et à mesure des séances.
La hierarchie des fichiers est la suivante :

— `TB` : contient les fichiers sources du testbench, le script python et le code C pour l'interface avec le testbench.
— `rtl` : contient les fichiers sources du circuit matériel.
— `build.sh` : script de compilation et de lancement de la simulation.

Conservez cette hierarchie de fichiers pour le bon fonctionnement. En effet les scripts de compilation et de lancement de la simulation sont écrits pour fonctionner avec cette hierarchie. De même d'un TD à l'autre si vous voulez avoir un dossier different par TD, gardez cette hierarchie pour chacun d'eux et copiez les fichiers nécessaires dans le nouveau dossier.

Ci dessous, vous trouverez un bloc diagramme (Figure :1) ainsi qu'un chromatogramme (Figure :2) décrivant le fonctionnement de cet AES.

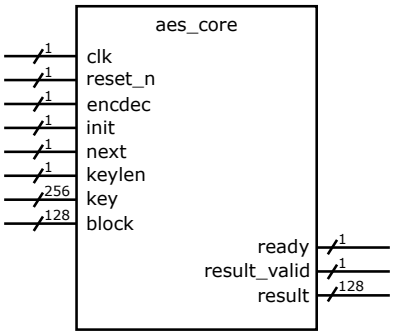


FIGURE 1 – aes_core

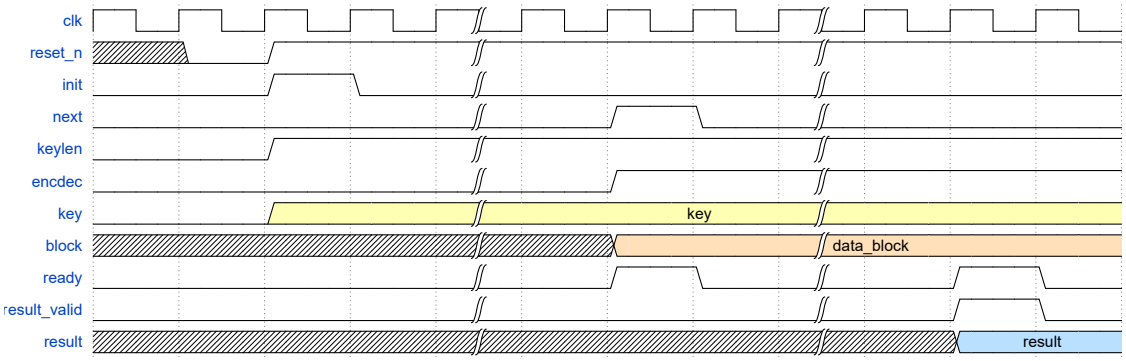


FIGURE 2 – Chronogramme de l'aes_core

Dans le répertoire *tb* vous trouverez le code squelette du test écrit en langage SystemVerilog : *aes_tb.sv*. Vous allez devoir compléter ce code.

Câblage

En fin du fichier *aes_tb.sv* se trouve l'instanciation de l'AES :

```
aes_cipher_top uut(.clk(tb_clk),
                   .reset_n(tb_reset_n),
                   .encdec(...),
                   .init(...),
                   .next(...),
                   .ready(...),
```

```
.key(...),
.keylen(...),
.block(...),
.result(...),
.result_valid(...)
          );
```

Listing 1 – instanciation de l'AES

Complétez le câble de l'instance de l'AES en vous aidant des variables de type *(*[logic]) ayant le prefixe *tb_* déclarées en tête du fichier.

Génération d'un stimuli pour la simulation d'un chiffrement

Le bloc *initial* contient pour l'instant uniquement la génération du reset. Inspirez vous de ce qui a été fait pour le reset de manière à simuler le chiffrement d'un texte clair de 128 bits. On pourra prendre par exemple un des vecteurs de test proposé dans la FiPS197 (`https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf`) :

— key=0x2b7e151628aed2a6abf7158809cf4f3c
— text_in=0x3243f6a8885a308d313198a2e0370734
— text_out=0x3925841d02dc09fbdc118597196a0b32

On rappelle que :
— @(posedge signal) permet d'attendre un front montant de *signal*
— # 0.5 ; permet d'attendre 0.5 unité de temps défini par la directive *timescale*
— les conventions lexicales pour les formats numériques sont les suivantes :
  — <taille>'<base><nombre>
  — 'b (binaire) :[01xXzZ]
  — 'd (decimal) :[0123456789]
  — 'o (octal) :[01234567xXzZ]
  — 'h (hexadecimal) peut être séparé par _ A5A5_FFFF
— La tache système *$display* avec le format *"%h"* permet d'afficher le contenu d'une variable en hexadécimal

Dans le fichier *aes_tb.sv*, des tâches (*task*) vides sont données, utilisez-les pour creer les stimulis nécessaires au fonctionnement de l'AES.

Vérifiez sur le chronogramme affiché par le simulateur que le texte chiffré est le bon.

4 Analyse du desisgn & introspection

Dans cette partie du TD nous allons voir comment il est possible d'instrospecter l'intérieur de l'AES sans modifier son interface. L'instrosection est très utile durant la phase de mise au point mais elle peut s'avérer également utile pour éléborer des tests plus complets dans le but , comme nous le verrons dans le TD3, d'amélorier par exemple la couverture de code.

Dans un contexte industriel, le designer et l'ingenieur de verification ne sont pas necessairement la même personne, ceci impose un part d'analyse du design à tester.
Pour cet AES, nous savons que l'état, c'est-à-dire les resultats intermediaires des calculs, est stocké dans des registres et que chaque ronde comporte 5 cycles d'horloge.

L'objectif ici est d'identifier les registres de l'état d'observer leurs valeurs avec l'introspection.

Pour réaliser cette introspection, déclarez les variables de type *logic* suivantes :
w0_r,w1_r,w2_r,w3_r

Effectuez le câblage tel que décrit sur la Figure : 3 en les reliant aux signaux identifiés comme étant les registres d'état.

On rappelle qu'une affectation continue se fait à l'aide de *assign* et qu'il suffit de donner le chemin hiérarchique, en prenant comme séparateur hiérarchique le ., pour accéder à une variable interne.

Affichez la valeur des 4 vecteurs lignes de l'AES durant l'exécution de celui-ci. On pourra par exemple réaliser une boucle *for* juste après l'activation du signal *next* et qui progressera à chaque coup d'horloge *clk_r* en s'aidant des signaux *sword_ctr_reg* et *round_ctr_reg* du module aes_encipher_block.

Cette convention d'affichage de l'état interne de l'AES est conforme à celle proposée dans le FIPS197.

A l'aide du fichier excel :`https://www.nayuki.io/page/aes-cipher-internals-in-excel` , vérifiez que votre introspection est correcte.

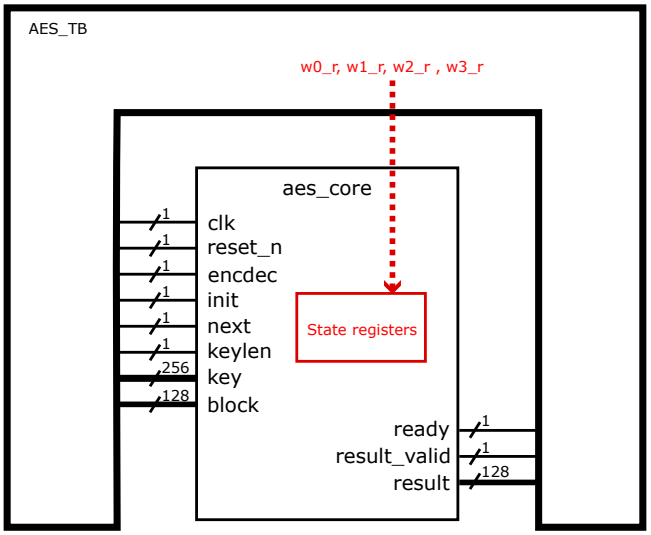


FIGURE 3 – introspection

5 Génération des vecteurs de test

Nous allons dans la suite du TD utiliser des vecteurs de test issus de fichiers texte.
Il nous faut donc, dans un premier temps, générer ces fichiers textes. Nous allons pour cela nous aider du Language *Python* et de la librairie *pycryptodome* dont la documentation se trouve là : `https://pycryptodome.readthedocs.io/en/latest/src/cipher/cipher.html`

Créez un script python que vous nommerez *gen_aes_test_vectors.py*.

Importez la librairie *AES* du module *Crypto.Cipher* ainsi que la fonction *get_random_bytes* du module *Crypto.Random*.

Dans un premier, utilisez l'exemple tiré de la FIPS197 utilisé dans section 2.

Créez une clé de chiffrement, *key*, qui doit être une séquence d'octets *bytes* et initialisez là avec la valeur proposée dans la section2. On rappelle que la méthode *fromhex* de l'objet *bytes* permet de créer et d'initialiser une séquence d'octets à partir d'une chaîne de caractères

Créez une instance de l' objet *AES* initialisé avec *key* en mode *AES_MODE_ECB* que vous nommerez *Cipher*

Créez et initialisez une séquence d'octets pour le texte clair que vous nommerez *pt*.

Récupérez le texte chiffré *ct* sous forme d'une séquence d'octets en faisant appel à la méthode *encrypt* de l'instance *Cipher* de l'objet *AES*..

Affichez le résultat du chiffrement. On rappelle que la méthode *hex* de l'objet *bytes* permet de récupérer la chaîne de caractères correspondant à la conversion hexadécimale de la séquences d'octets.

A ce stade vous savez réaliser le chiffrement d'un texte clair. Pour générer des textes clairs aléatoirement nous utilisons la fonction *get_random_bytes*. Vous êtes maintenant en mesure de créer deux fichiers :

- data_in.txt, dans lequel vous placerez 100 textes clairs aléatoires
- data_out_ref.txt, dans lequel vous placerez les 100 textes chiffrés correspondants

Vérifiez succintement le contenu des fichiers générés à l'aide d'un éditeur de texte.

6 test SV

Complétez le bloc *initial* du test *aes_tb.sv* à l'aide d'une boucle *while* dans laquelle vous lirez un à un les textes clairs du fichier *data_in.txt* dont vous effectuerez le chiffrement à l'aide du bloc matériel AES et vous placerez les textes chiffrés ainsi obtenu dans un ficher nommé *data_out.txt*.
On rappelle que :

- la tâche *$fopen* permet d'ouvrir un fichier dans le mode désiré.
  int fd=$fopen("filename","mode") ("filename") est le chemin vers le fichier à ouvrir. *fd* est le descripteur de fichier auquel on fait référence pour toutes les opérations qui seront effectuées sur ce fichier.
  "mode" est une chîne de caractère qui spécifie dans quel mode le fichier est ouvert.

  - "r" Open for reading
  - "w" Create for writing, overwrite if it exists
  - "a" Create if file does not exist, else append ; open for writing at end of file
  - "r+" Open for update (reading and writing)
  - "w+" Truncate or create for update
  - "a+" Append, open or create for update at EOF

- *$fclose* permet de fermer le fichier : *$fclose(fd)*

- *$feof* renvoie *true* si la fin du fichier a été atteinte.

- *$fgets* permet de récupérer une ligne dans une *string* : *$fgets(s,fd)*

- *$fwrite* permet d'écrire une chaîne de caractère formatée dans un fichier : *$fwrite(fd,"format string", variable)*

- *$sscanf* est l'équivalent du sscanf de la librairie C. Il vous est conseillé de retirer les caractères de fin de ligne *CR* ou *LF* de la *string* à traiter à l'aide de *sscanf*, on pourra utiliser par exemple la méthode *substr* de l'objet SystemVerilog *string* pour n'extraire que la partie informative de la ligne de texte lue.

Une fois le test effectué comparez le fichier généré par le test *data_out.txt* avec le fichier de référence généré dans la section précédente *data_out_ref.txt*.

Modifiez le test pour effectuez la vérification du texte chiffré directement dans le test écrit en SystemVerilog.